MY WORK DOCUMENTATION

Apollo AgriVerse

My Work Documentation — Crop Dataset • NASA POWER API • FastAPI • Frontend Integration

Weather / API Development

Dataset Documentation

Digital Twin Support

Nandini Naresh Naral

Contents

01

APOLLO AGRIVERSE · WORK DOCUMENTATION

Project Overview

PROJECT

Apollo AgriVerse

Main focus: Crop dataset documentation, weather dataset checking, NASA POWER API testing, and FastAPI weather API development.

MY SCOPE

Weather / API side and related dataset documentation/testing

ML model training and integration are handled by a teammate.

02

APOLLO AGRIVERSE · WORK DOCUMENTATION

My Work Flow

Crop Dataset

↓

Understand structure

↓

Cleaning / preprocessing documentation

↓

Weather Dataset

↓

Check raw & cleaned data

↓

Build local weather service

↓

Test NASA POWER API

↓

Create NASA weather service

↓

Create FastAPI application

↓

Prepare /weather endpoint

↓

Frontend integration

APOLLO AGRIVERSE • WORK DOCUMENTATION

03

Crop Dataset

The crop recommendation dataset contains soil and environmental information.

Feature	Meaning
N	Nitrogen
P	Phosphorus
K	Potassium
temperature	Air temperature (°C)
humidity	Relative humidity (%)
ph	Soil pH
rainfall	Rainfall (mm)
label	Crop name / target

Documented Pipeline

Raw Dataset → **Data Cleaning** → **Feature Engineering** → **Train-Test Split** → **Feature Scaling** → **Processed Dataset**

Folder Structure

```
02_Datasets/
├── Raw/Crop/Crop_recommendation.csv
├── Interim/Crop/crop_clean.csv
└── Processed/Crop/
    ├── X_train_processed.csv
    ├── X_test_processed.csv
    ├── y_train.csv
    └── y_test.csv

06_ML/notebooks/
├── crop_data_cleaning.ipynb
└── crop_preprocessing.ipynb
```

> The raw dataset is not modified. Processing is performed on copies.

Cleaning & Preprocessing

- Remove duplicate records
- Check missing values
- Verify data types
- Remove invalid values
- Standardize column names
- Separate input/output
- Train-test split
- Feature scaling

APOLLO AGRIVERSE · WORK DOCUMENTATION

04

Crop AI Model

Grape Crop Stage Classification Model

Notebook: `06_ML/notebooks/train_crop_model.ipynb`

Model: `06_ML/models/grape_crop_stage_model.keras`

Purpose: Classifies grape images into 5 growth stages — Bud Burst, Vegetative, Flowering, Fruit Development, and Harvest.

03

APOLLO AGRIVERSE • WORK DOCUMENTATION

Weather API Folder Structure

```
05_Backend/
├── app/
│   └── main.py
├── api/
│   └── weather_api/
│       ├── nasa_api.py
│       ├── test_nasa_weather.py
│       ├── weather_service.py
│       └── check_weather_rows.py
└── venv/
```

File	Purpose
main.py	Main FastAPI application
nasa_api.py	NASA POWER weather request function
test_nasa_weather.py	Tests NASA POWER connectivity
weather_service.py	Loads and reads the local weather dataset
check_weather_rows.py	Used during weather dataset checking

04

APOLLO AGRIVERSE • WORK DOCUMENTATION

What Is an API?

An API allows one software system to request data from another system.

Frontend

↓

FastAPI Weather API

↓

NASA POWER / Local Dataset

↓

JSON Weather Response

↓

Frontend

Example request:

```
/weather?latitude=17.66&longitude=75.91
```

03 APOLLO AGRIVERSE • WORK DOCUMENTATION

NASA POWER API Work

NASA POWER was tested before connecting it to FastAPI.

Solapur, Maharashtra
Latitude 17.66° | Longitude 75.91°

✓ Status Code: 200 — API connection worked.

04

APOLLO AGRIVERSE • WORK DOCUMENTATION

NASA POWER Hourly API

Parameter	Meaning
T2M	Temperature at 2 meters
RH2M	Relative humidity at 2 meters
PRECTOTCORR	Corrected precipitation

NASA returned hourly values. Example temperature values included **22.67, 22.42, 22.25, ..., 31.35, ..., 23.29**. Humidity and precipitation were also returned hourly.

03

APOLLO AGRIVERSE • WORK DOCUMENTATION

Handling NASA -999.0

⚠ **Important:** NASA returned `-999.0` for some requested data. This is a NASA POWER fill/missing-data value, not a real weather measurement.

The application must check these values before sending data to the frontend.

04

APOLLO AGRIVERSE • WORK DOCUMENTATION

Clean NASA Weather Response

Instead of returning the complete NASA response, the service returns only required values.

```json
{
  "source": "NASA POWER",
  "latitude": 17.66,
  "longitude": 75.91,
  "time": "2026080923",
  "temperature_c": 23.29,
  "humidity_pct": 88.12,
  "rainfall_mm": 0.0
}
```

> A smaller response is easier for the frontend to consume.

03

APOLLO AGRIVERSE • WORK DOCUMENTATION

How nasa_api.py Works

Latitude + Longitude

↓

nasa_api.py

↓

NASA POWER Hourly API

↓

NASA JSON

↓

Read T2M / RH2M / PRECTOTCORR

↓

Select required hour

↓

Clean weather JSON

Main function: `get_nasa_weather(latitude, longitude)`

APOLLO AGRIVERSE • WORK DOCUMENTATION

04

FastAPI Work

FastAPI was used to create the REST API layer.

```
from fastapi import FastAPI

app = FastAPI(
    title="Apollo AgriVerse API",
    description="Backend API for Intelligent Soil & Grape Digital Twin",
    version="1.0.0"
)
```

Main application: `05_Backend/app/main.py`

APOLLO AGRIVERSE • WORK DOCUMENTATION

03

Running FastAPI

1. `.\venv\Scripts\Activate.ps1`
2. `uvicorn app.main:app --reload`

✓ Server successfully started at `http://127.0.0.1:8000`

APOLLO AGRIVERSE • WORK DOCUMENTATION

04

Swagger / OpenAPI

/docs Interactive FastAPI documentation

```
http://127.0.0.1:8000/docs
```

✓ GET / → 200 OK

APOLLO AGRIVERSE • WORK DOCUMENTATION

03

India-Wide Weather

The API is designed for different Indian locations instead of only one city.

Location	Latitude	Longitude
Solapur	17.66	75.91
Pune	18.52	73.85
Nashik	19.99	73.78
Nagpur	21.15	79.09

Intended India boundary: approximately 6.0°N–37.5°N latitude and 68.0°E–97.5°E longitude.

04

APOLLO AGRIVERSE • WORK DOCUMENTATION

Why Latitude and Longitude?

City selected

↓

Latitude + Longitude

↓

Weather API

↓

Weather for that geographic point

This means a separate endpoint is not needed for every city.

15

APOLLO AGRIVERSE • WORK DOCUMENTATION

Weather API Focus

My work on the weather side focused on **NASA POWER API testing and FastAPI integration**, not on creating, cleaning, or validating the weather dataset.

Frontend

↓

FastAPI /weather

↓

NASA POWER API

↓

Clean JSON Weather Response

↓

Frontend / Digital Twin

> **Scope clarification:** Weather dataset creation, cleaning, preprocessing, and row-level validation are not listed as my work.

APOLLO AGRIVERSE · WORK DOCUMENTATION

04 Current Architecture

APOLLO AGRIVERSE

↓

User selects location

↓

Latitude + Longitude

↓

FastAPI /weather

NASA POWER **Online source**	**Local CSV** **Offline source**

↓

Clean JSON

↓

Frontend

↓

Grape Digital Twin

> The broader target architecture describes Open-Meteo as primary, NASA POWER as secondary, and local CSV as fallback. Only completed implementations should be presented as completed; the tested work covered NASA POWER and local CSV.

03

APOLLO AGRIVERSE • WORK DOCUMENTATION

My Completed Work

- ✓ Studied the crop dataset structure.
- ✓ Documented crop dataset locations and features.
- ✓ Focused on the weather API integration and online weather service.
- ✓ Tested the weather API and its responses.

- ✓
- ✓
- ✓
- ✓ Tested NASA POWER connectivity.
- ✓ Tested NASA POWER Hourly API.
- ✓ Tested Indian coordinates.
- ✓ Identified NASA `-999.0` fill values.
- ✓ Reduced NASA data to required weather parameters.
- ✓ Created a clean NASA weather response.
- ✓ Created the FastAPI application.
- ✓ Installed/fixed required Python dependencies.
- ✓ Started FastAPI successfully.
- ✓ Tested the root API endpoint.
- ✓ Prepared the `/weather` endpoint for frontend integration.

APOLLO AGRIVERSE • WORK DOCUMENTATION

04

What I Am Not Doing

ML model training

ML model integration

Crop prediction model deployment

These are handled by my teammate.

My main responsibility: weather/API side and related dataset documentation/testing.

03

APOLLO AGRIVERSE • WORK DOCUMENTATION

Final API Example


```json
{
  "source": "NASA POWER",
  "latitude": 17.66,
  "longitude": 75.91,
  "time": "2026080923",
  "temperature_c": 23.29,
  "humidity_pct": 88.12,
  "rainfall_mm": 0.0
}
```


23.29 °C	88.12 %	0.0 mm	17.66, 75.91
Temperature	Humidity	Rainfall	Location

APOLLO AGRIVERSE · WORK DOCUMENTATION

04

Tools Used

Tool / Technology	Purpose
Python	Programming
Pandas	Dataset loading/checking
Requests	NASA POWER API requests
FastAPI	REST API
Uvicorn	FastAPI server
NASA POWER	Online weather source
CSV	Historical/offline weather source
Swagger/OpenAPI	API testing
VS Code	Development
Git/GitHub	Version control

23

APOLLO AGRIVERSE • WORK DOCUMENTATION

Short Presentation Explanation

My work was mainly on the crop dataset documentation and weather API side. I understood and documented the crop dataset structure, then tested NASA POWER as an online weather source. I created a Python NASA service that takes latitude and longitude and returns temperature, humidity and rainfall. Then I created a FastAPI application so the frontend can request weather data through an API endpoint. I also tested the API using Swagger. I did not work on the weather dataset creation, cleaning or preprocessing, and the ML model integration is being handled by my teammate.

04

APOLLO AGRIVERSE • WORK DOCUMENTATION

Important Source Notes

Source distinction: The crop documentation states that the raw crop dataset is kept unchanged and preprocessing is performed on copies.

Weather architecture: The project weather manual describes an India-focused agricultural service with live telemetry, NASA POWER backup, local CSV fallback, validation, sanitization, caching and frontend integration.

Documentation rule: Completed items should be distinguished from future/planned architecture.